

BDBT – Obsługa perspektyw użytkowników

Tomasz Mrozek
Instytut Telekomunikacji
Wydział Elektroniki i Technik Informatycznych



Zakres prezentacji

- Obsługa perspektyw dla użytkowników admin i user

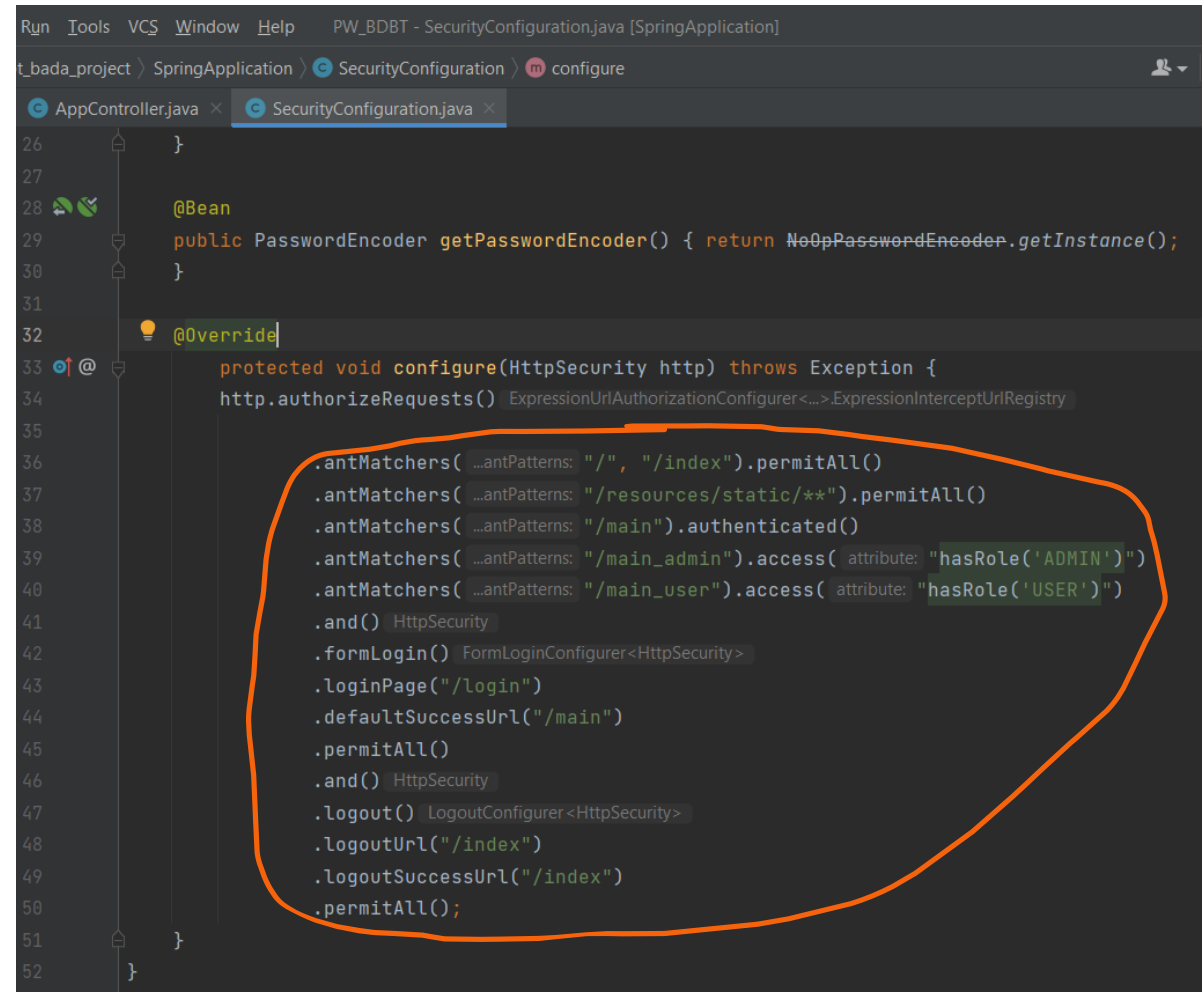
1

OBSŁUGA PERSPEKTYW

Obsługa perspektyw

Przejsć do pliku **SecurityConfiguration.java** do metody **configure**.

Dokonać modyfikacji istniejącego kodu zgodnie z tym zamieszczonym na rysunku obok.



```
Run Tools VCS Window Help PW_BDBT - SecurityConfiguration.java [SpringApplication]
t_bada_project > SpringApplication > SecurityConfiguration > configure
AppController.java x SecurityConfiguration.java x
26 }
27
28 @Bean
29 public PasswordEncoder getPasswordEncoder() { return NoOpPasswordEncoder.getInstance();
30 }
31
32 @Override
33 protected void configure(HttpSecurity http) throws Exception {
34     http.authorizeRequests() ExpressionUrlAuthorizationConfigurer<...>.ExpressionInterceptUrlRegistry
35
36     .antMatchers( ...antPatterns: "/", "/index").permitAll()
37     .antMatchers( ...antPatterns: "/resources/static/**").permitAll()
38     .antMatchers( ...antPatterns: "/main").authenticated()
39     .antMatchers( ...antPatterns: "/main_admin").access( attribute: "hasRole('ADMIN')" )
40     .antMatchers( ...antPatterns: "/main_user").access( attribute: "hasRole('USER')" )
41     .and() HttpSecurity
42     .formLogin() FormLoginConfigurer<HttpSecurity>
43     .loginPage("/login")
44     .defaultSuccessUrl("/main")
45     .permitAll()
46     .and() HttpSecurity
47     .logout() LogoutConfigurer<HttpSecurity>
48     .logoutUrl("/index")
49     .logoutSuccessUrl("/index")
50     .permitAll();
51 }
52 }
```

Obsługa perspektyw

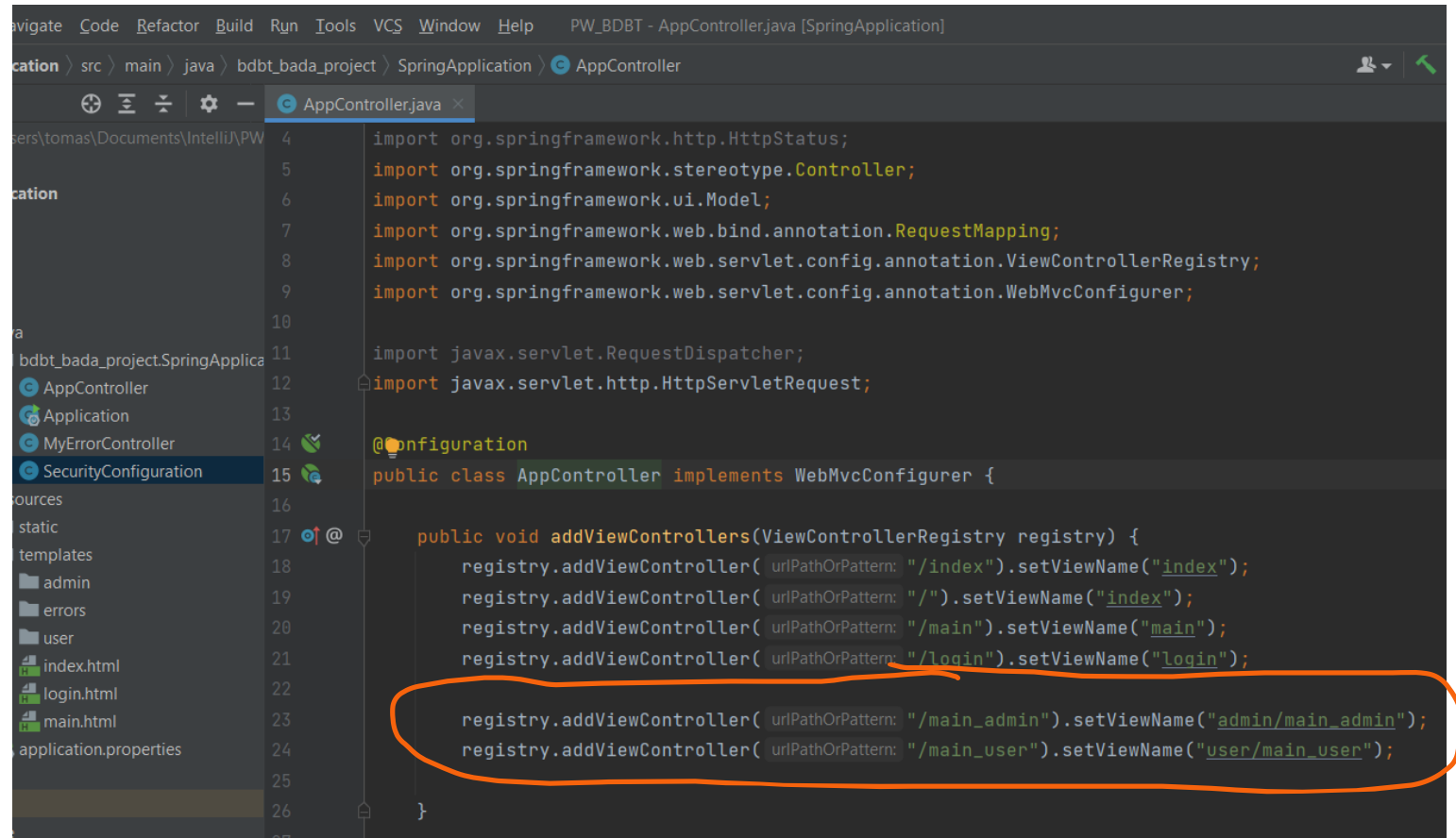
Fragment kodu dla pliku **SecurityConfiguration.java** (do uaktualnienia/modyfikacji):

```
.antMatchers("/", "/index").permitAll()
.antMatchers("/resources/static/**").permitAll()
.antMatchers("/main").authenticated()
.antMatchers("/main_admin").access("hasRole('ADMIN')")
.antMatchers("/main_user").access("hasRole('USER')")
.and()
.formLogin()
.loginPage("/login")
.defaultSuccessUrl("/main")
.permitAll()
.and()
.logout()
.logoutUrl("/index")
.logoutSuccessUrl("/index")
.permitAll();
```

Obsługa perspektyw

Przejsć do pliku **AppController.java**

Dodać w pliku zaznaczony fragment kodu.



```
4 import org.springframework.http.HttpStatus;
5 import org.springframework.stereotype.Controller;
6 import org.springframework.ui.Model;
7 import org.springframework.web.bind.annotation.RequestMapping;
8 import org.springframework.web.servlet.config.annotation.ViewControllerRegistry;
9 import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
10
11 import javax.servlet.RequestDispatcher;
12 import javax.servlet.http.HttpServletRequest;
13
14 @Configuration
15 public class AppController implements WebMvcConfigurer {
16
17     @Override
18     public void addViewControllers(ViewControllerRegistry registry) {
19         registry.addViewController("/index").setViewName("index");
20         registry.addViewController("/").setViewName("index");
21         registry.addViewController("/main").setViewName("main");
22         registry.addViewController("/login").setViewName("login");
23         registry.addViewController("/main_admin").setViewName("admin/main_admin");
24         registry.addViewController("/main_user").setViewName("user/main_user");
25     }
26 }
```

Obsługa perspektyw

Fragment kodu dla pliku **AppController.java** (do skopiowania):

```
registry.addViewController("/main_admin").setViewName("admin/main_admin");  
registry.addViewController("/main_user").setViewName("user/main_user");
```

Obsługa perspektyw

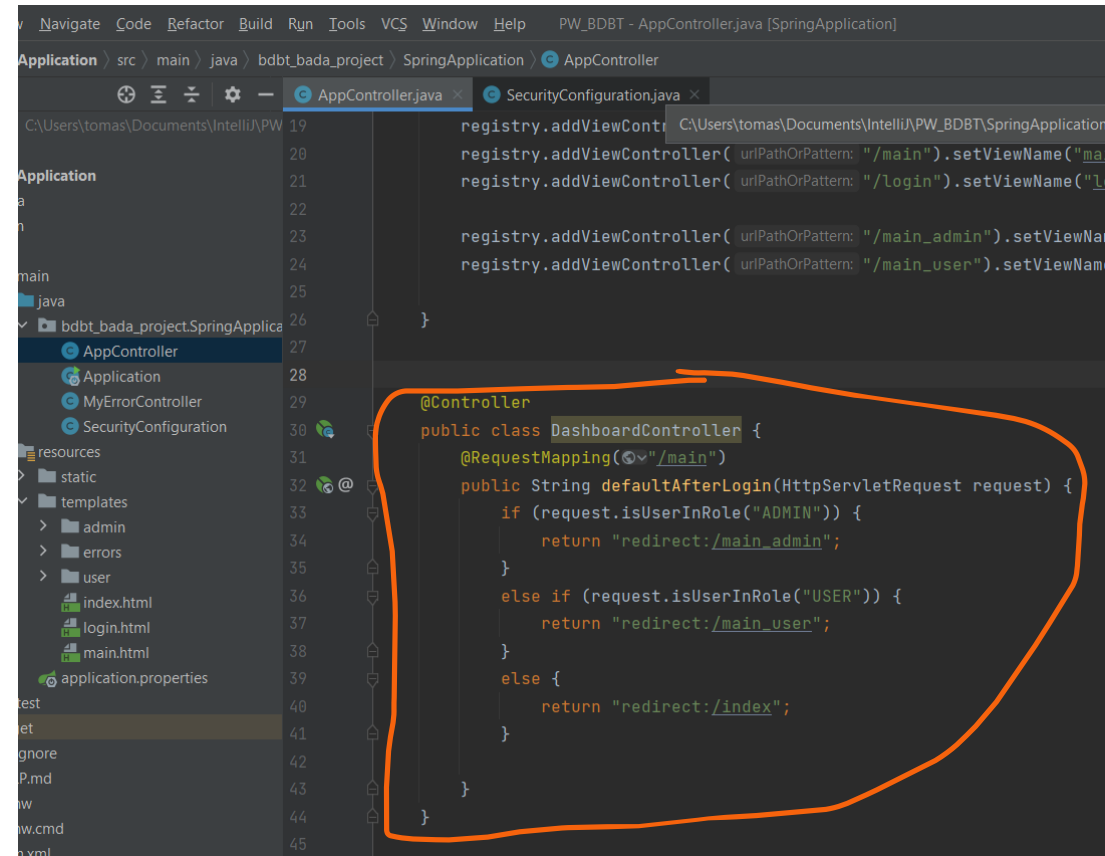
W tym samym pliku **AppController.java** dodać kontroler odpowiedzialny za wyświetlanie stron po zalogowaniu na odpowiednie konto.

Należy pamiętać o „automatycznym dodaniu” odpowiednich adnotacji tzn.:

@Controller,

@RequestMapping,

oraz importować bibliotekę dla podkreślonego na czerwono fragmentu HttpServletRequest.



```
19 registry.addViewContr
20 registry.addViewContr
21 registry.addViewContr
22
23 registry.addViewContr
24 registry.addViewContr
25
26 }
27
28
29
30 @Controller
31 public class DashboardController {
32     @RequestMapping("/main")
33     public String defaultAfterLogin(HttpServletRequest request) {
34         if (request.isUserInRole("ADMIN")) {
35             return "redirect:/main_admin";
36         }
37         else if (request.isUserInRole("USER")) {
38             return "redirect:/main_user";
39         }
40         else {
41             return "redirect:/index";
42         }
43     }
44 }
45
```


Obsługa perspektyw

Fragment kodu dla pliku **AppController.java** (do skopiowania):

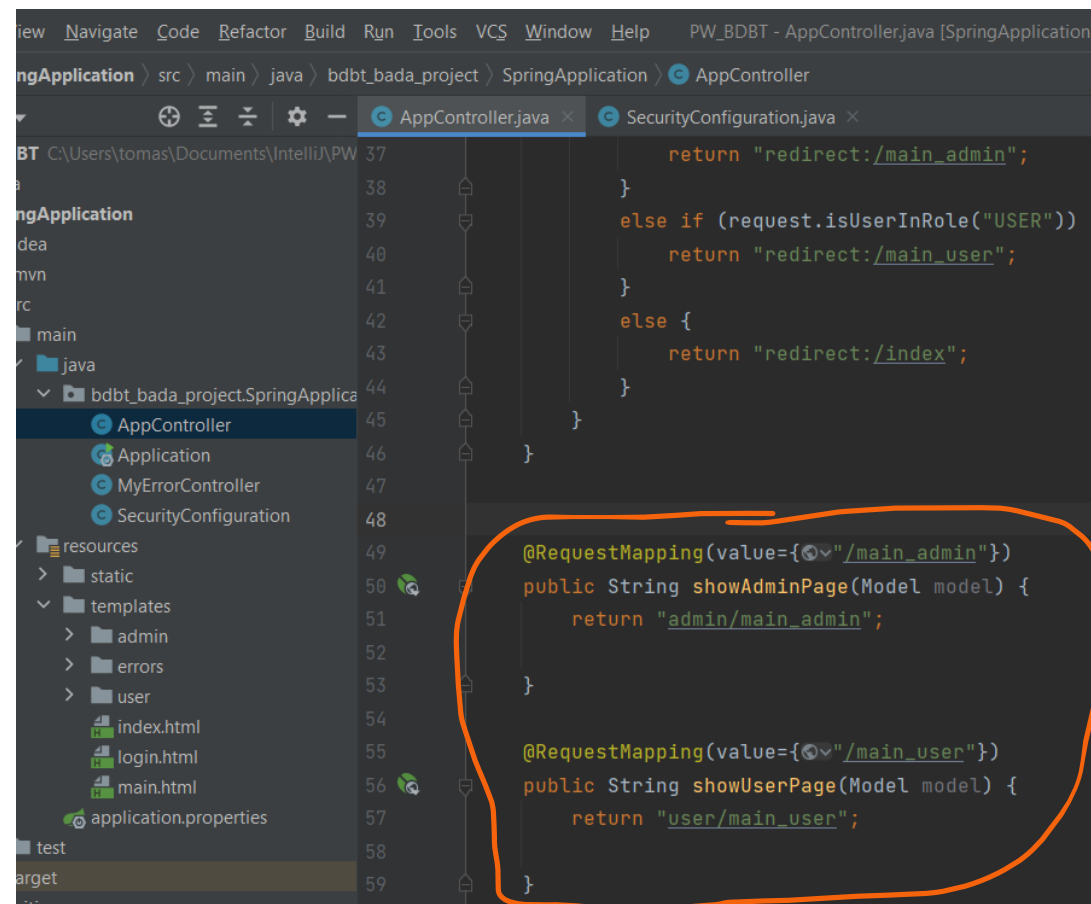
```
@Controller
public class DashboardController {
    @RequestMapping("/main")
    public String defaultAfterLogin(HttpServletRequest request) {
        if (request.isUserInRole("ADMIN")) {
            return "redirect:/main_admin";
        }
        else if (request.isUserInRole("USER")) {
            return "redirect:/main_user";
        }
        else {
            return "redirect:/index";
        }
    }
}
```

Obsługa perspektyw

W tym samym pliku **AppController.java** należy następnie dodać kolejny fragment kodu.

Poprawnie w pisaniu kodu najpierw pisze się metodę, a następnie dodaje adnotację i importuje biblioteki.

Uwaga: Po ręcznym dodaniu adnotacji `@RequestMapping` może być potrzeba użycia automatycznej opcji importowania biblioteki na początku kodu. W tym miejscu nie ma takiej potrzeby, ponieważ biblioteka dla tej adnotacji została już wcześniej zaimportowana.



```
view  Navigate  Code  Refactor  Build  Run  Tools  VCS  Window  Help  PW_BDBT - AppController.java [SpringApplication]
ngApplication > src > main > java > bdbt_bada_project > SpringApplication > AppController
AppController.java x SecurityConfiguration.java x
BT C:\Users\tomas\Documents\IntelliJ\PW 37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59

return "redirect:/main_admin";
}
else if (request.isUserInRole("USER"))
return "redirect:/main_user";
}
else {
return "redirect:/index";
}
}
}

@RequestMapping(value={"/main_admin"})
public String showAdminPage(Model model) {
return "admin/main_admin";
}

@RequestMapping(value={"/main_user"})
public String showUserPage(Model model) {
return "user/main_user";
}
```

Obsługa perspektyw

Fragment kodu dla pliku **AppController.java** (do skopiowania):

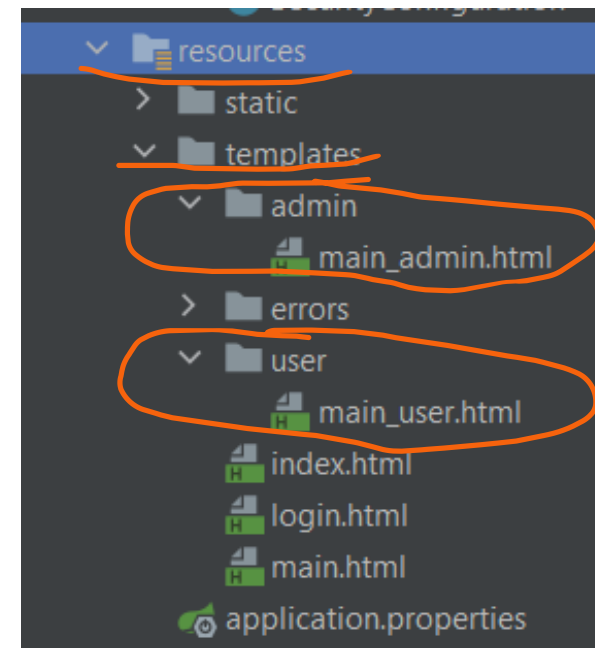
```
@RequestMapping(value={"/main_admin"})  
public String showAdminPage(Model model) {  
    return "admin/main_admin";  
}  
  
@RequestMapping(value={"/main_user"})  
public String showUserPage(Model model) {  
    return "user/main_user";  
}
```

Obsługa perspektyw

Kolejnym etapem jest stworzenie folderów **admin** oraz **user**, a w nich plików html odpowiednio **main_admin.html** oraz **main_user.html**. Stworzenie folderów pozwoli nam na zachowanie porządku w tworzonym projekcie.

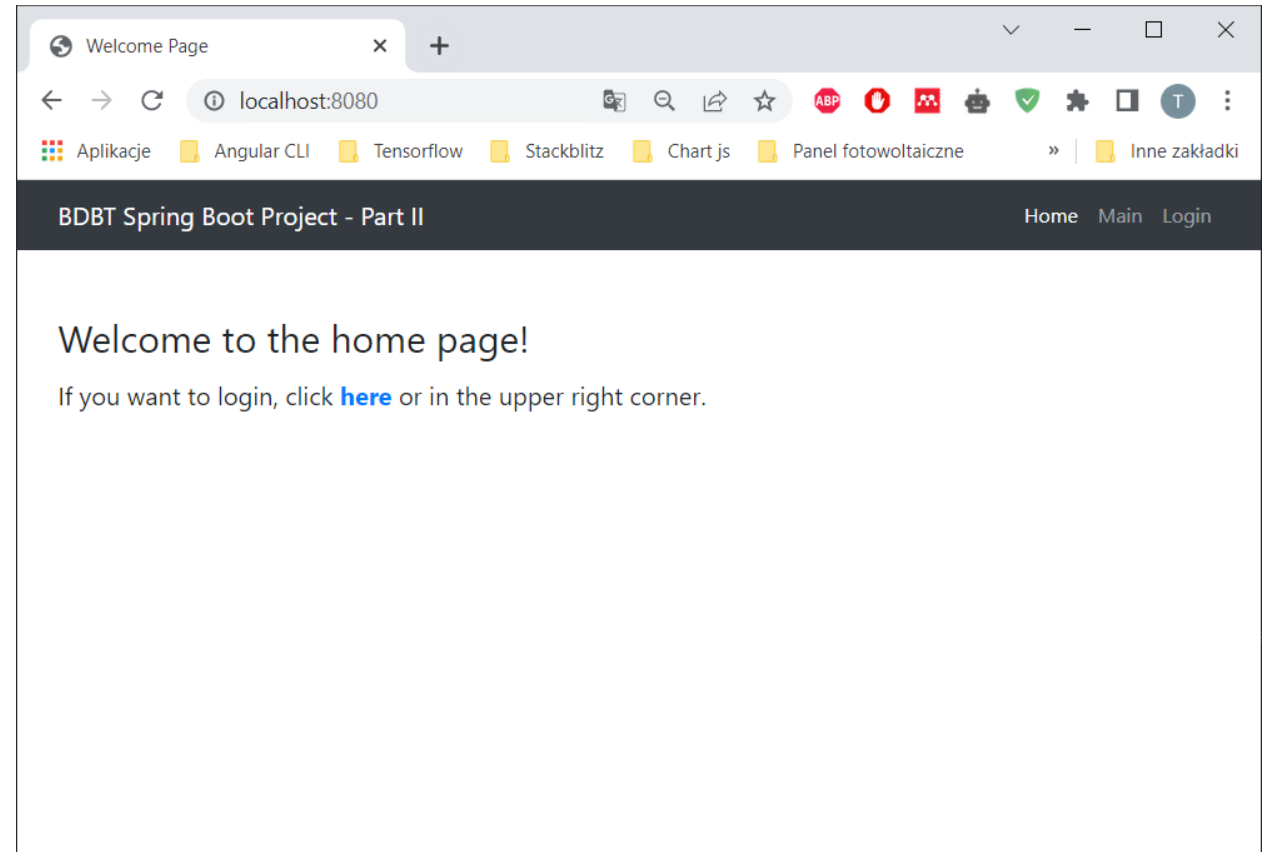
Strony te będą zawierały treść widoczną dla każdego użytkownika dostępną po zalogowaniu.

Zawartość plików .html zostanie udostępniona wraz z tą instrukcją w postaci pliku „ZałącznikiPerspektywy.7z”.



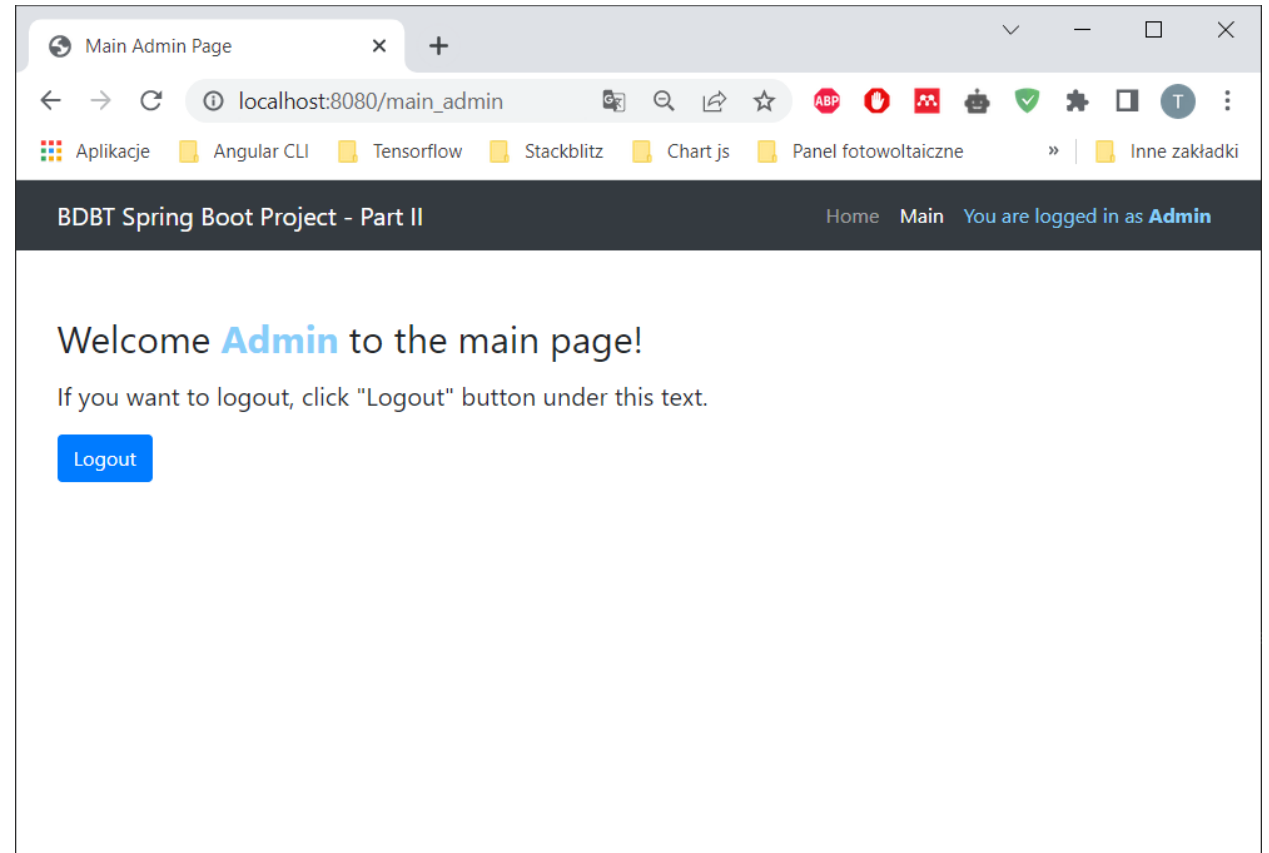
Obsługa perspektyw – wynik działania aplikacji

Po wpisaniu w adresie przeglądarki strony: **localhost:8080** należy zalogować się na perspektywę **admina** (login: admin, hasło: admin), albo **usera** (login: user, hasło: user).



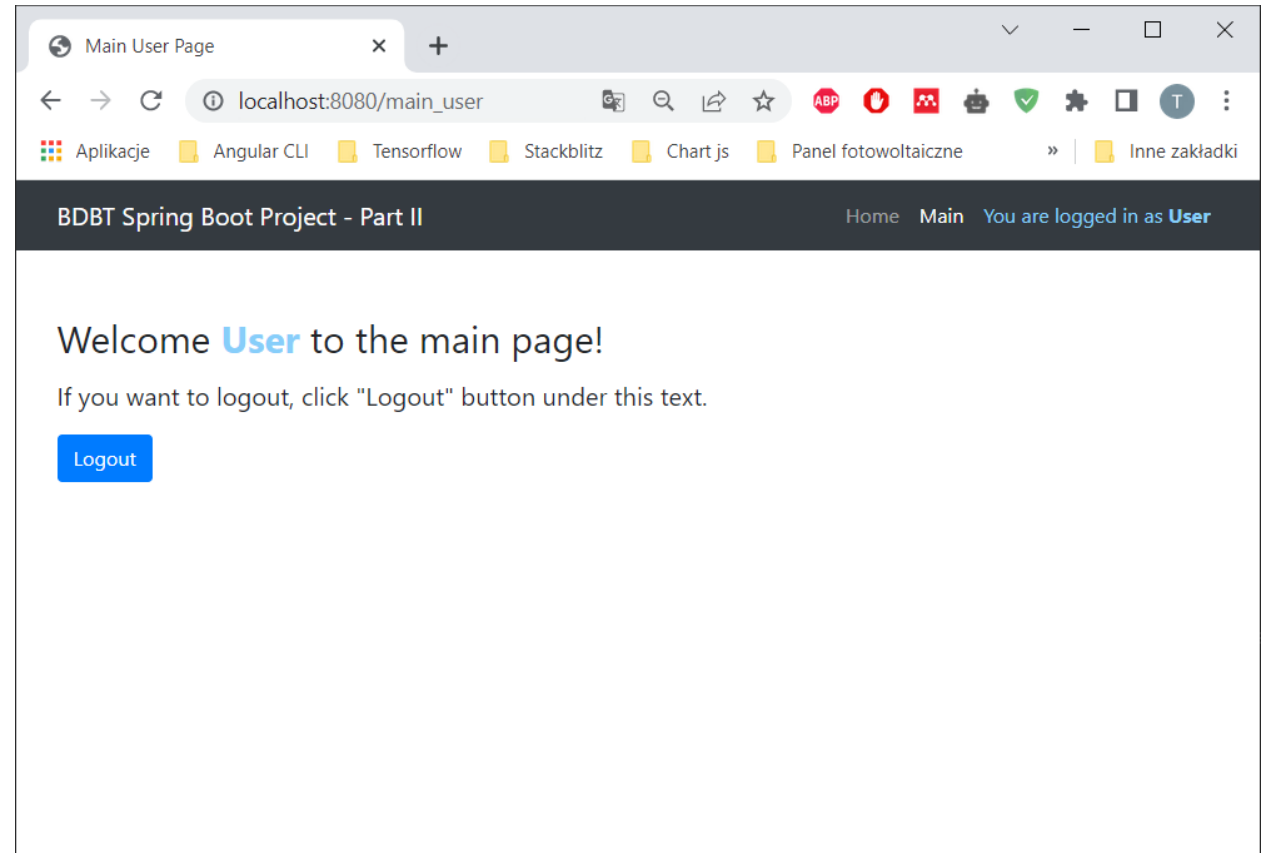
Obsługa perspektyw – wynik działania aplikacji

Widok perspektywy **admin**.



Obsługa perspektyw – wynik działania aplikacji

Widok perspektywy **user**.



Obsługa perspektyw – podsumowanie

Przeanalizować kod oraz dostosować go do swojego rozwiązania (aplikacji). Dla stworzonych perspektyw należy stworzyć kod umożliwiający wykonywanie odpowiednich operacji bazodanowych (wyświetlanie odpowiedni tabel dla danej perspektywy wraz z wymaganymi operacjami typu CRUD).

**Politechnika
Warszawska**

Koniec

